

Exactly. **Persistence** is just a fancy technical word for:

Making data survive after the current request/code execution is over.

Let's connect it directly to Auth.js. 📌

Without persistence

Imagine Auth.js receives:

```
Google says:  
"This is Gopi."
```



Auth.js creates:

```
user = Gopi  
session = ABC123
```



But if those values only existed in memory:

```
Server memory  
-----  
user = Gopi  
session = ABC123
```



then the moment the server process restarts, that information could disappear.

So Auth.js needs somewhere to **persist** it — typically a database when using a database-backed setup.

With persistence

Auth.js → Adapter → Database:

```
Auth.js
  ↓
"Save this user"
  ↓
Adapter
  ↓
Database
```



Database:

```
User
-----
id: 123
name: Gopi
email: gopi@example.com
```



Now the data remains there after:

- the request finishes
- the user closes the browser
- the server restarts
- another server instance handles the next request

That's **persistence**.

In Auth.js, what does the adapter persist?

Depending on the configured database/session strategy, authentication-related records such as:

```
User
Account
Session
VerificationToken
```

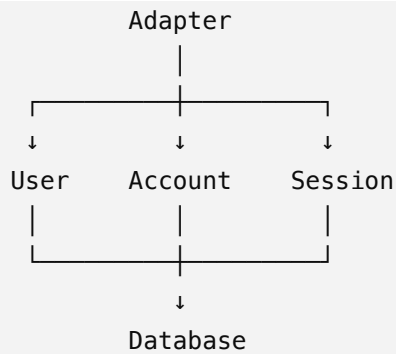


can be persisted.

For example:

```
Auth.js
  |
  ↓
```





Example

Google login succeeds.

Auth.js needs to remember:

Google account
↓
Application user
↓
Application session



The adapter allows Auth.js to save that information into your DB.

Think of it like a notebook 📓

Imagine Auth.js is a receptionist.

A user walks in:

"Hi, I'm Gopi."

The receptionist needs to remember her.

No persistence:

Receptionist thinks:

"Okay, Gopi is here."

Then the receptionist goes home. 😂

Tomorrow:

"Gopi who?"

With persistence:

Receptionist writes:

 User #123
Name: Gopi
Email: gopi@example.com



Tomorrow they can look in the notebook.

That notebook is your **database**.

The **adapter** is the person translating between the receptionist and the notebook:

Auth.js receptionist
↓
Adapter
↓
Database 



One important distinction

Persistence does NOT mean authentication itself.

The adapter isn't saying:

"Yes, Gopi's password is correct."

That's authentication logic.

Instead, it's saying:

"Okay, Auth.js, here's how you **save and retrieve** Gopi's authentication-related records from your database."

So:

Piece	Job
Google / OIDC	Authenticate/identify the user
Auth.js	Coordinate the auth/session flow
Adapter	Connect Auth.js to your DB
Database	Persist the records

Piece	Job
Cookie	Let the browser carry session information between requests

The simplest definition to remember:

Persistence = storing data somewhere durable so you can retrieve it later, instead of losing it when the current execution/request ends.

And "the adapter handles persistence" means:

The adapter gives Auth.js the ability to save and retrieve its users, accounts, sessions, etc. from your database.



okay

Perfect 😊

So your mental model now is:

```
Google / OIDC
  ↓
"Who is this user?"
  ↓
Auth.js
  ↓
Adapter
  ↓
Database
  ↓
Users / Accounts / Sessions persisted
  ↓
Session cookie
  ↓
Browser
```



And remember the key distinction:

Auth.js = authentication/session machinery

Provider = external identity system (e.g. Google)

Adapter = database bridge

Persistence = saving data so it can be retrieved later

Cookie = carries the session information between browser ↔ server

We can continue from here with the **exact Auth.js session lifecycle** next — **first login → user creation → account creation → session creation → cookie → subsequent request → `auth()` → session lookup → logout.** 🔥



yes

Perfect 😊 Let's do the **entire Auth.js session lifecycle**, but this time I want you to see **exactly what happens at every step and which code is yours vs Auth.js's internal code.**

Auth.js — Complete Flow Under the Hood

We'll use this setup:

```
Next.js
+
Auth.js / next-auth
+
Google (OIDC provider)
+
Prisma Adapter
+
PostgreSQL
```



The overall flow:

LOGIN



```
Browser
↓
Auth.js signIn("google")
↓
Google
↓
User authenticates
↓
Google callback
↓
Auth.js
```

↓
Adapter
↓
Database
↓
Session created
↓
Cookie
↓
Browser

LATER REQUEST

Browser
↓
Cookie automatically sent
↓
Next.js
↓
auth()
↓
Auth.js
↓
Adapter
↓
Database
↓
Current user/session
↓
Server Component

Let's go step by step.

1. Your configuration

You have:

</> TypeScript



// auth.ts

```
import NextAuth from "next-auth";  
import Google from "next-auth/providers/google";  
import { PrismaAdapter } from "@auth/prisma-adapter";  
  
import { prisma } from "@lib/prisma";
```

```
export const {
  handlers,
  auth,
  signIn,
  signOut,
} = NextAuth({
  adapter: PrismaAdapter(prisma),

  providers: [
    Google({
      clientId: process.env.AUTH_GOOGL_ID!,
      clientSecret: process.env.AUTH_GOOGL_SECRET!,
    }),
  ],
});
```

There are **two important configurations** here.

Provider

</> TypeScript



```
Google(...)
```

means:

"Google is an authentication provider for my application."

Adapter

</> TypeScript



```
PrismaAdapter(prisma)
```

means:

"Use Prisma to persist Auth.js data in my database."

So:

Google



authentication/identity

Prisma Adapter



database persistence



2. Route Handler

You also have:

</> TypeScript



```
// app/api/auth/[...nextauth]/route.ts

import { handlers } from "@auth";

export const { GET, POST } = handlers;
```

This is important because now requests such as:

```
/api/auth/signin
/api/auth/callback/google
/api/auth/signout
```



can be handled by Auth.js.

You aren't manually writing those authentication handlers.

Auth.js provides them.

3. User clicks "Continue with Google"

Your UI could be:

</> TypeScript



```
"use client";

import { signIn } from "next-auth/react";

export function LoginButton() {
  return (
    <button onClick={() => signIn("google")}>
      Continue with Google
    </button>
  );
}
```

The browser executes:

</> TypeScript



```
signIn("google")
```

Conceptually:

```
Browser
  ↓
"Start Google login"
  ↓
Auth.js endpoint
```



4. Auth.js redirects to Google

Auth.js knows from your configuration:

```
</> TypeScript

providers: [
  Google(...)
]
```



So it creates the appropriate OAuth/OIDC authorization request.

Then:

```
Browser
  ↓
Google
```



The browser is now on Google's website.

5. Google authenticates the user

Google handles:

```
Email
Password
MFA
etc.
```



Your application **doesn't** receive the Google password.

That's important.

Gopi's Google password



Google



Not:

Gopi's Google password



Your Next.js ❌



6. Google redirects back

After successful authentication, Google redirects the browser back to your callback:

GET /api/auth/callback/google?code=ABC123



Now the request enters your Next.js server.

Browser



Next.js Route Handler



Auth.js



Remember this code?

`</> TypeScript`

```
export const { GET, POST } = handlers;
```



That's what connects the URL to Auth.js's implementation.

7. Auth.js processes the callback

Auth.js sees:

```
provider = google  
code = ABC123
```



It processes the OAuth/OIDC callback.

Conceptually:

```
authorization code
  ↓
Auth.js
  ↓
Google
  ↓
token / identity information
```



Auth.js can now determine the Google identity associated with the login.

For example, conceptually:

```
</> JSON

{
  "provider": "google",
  "providerAccountId": "987654321",
  "email": "gopi@example.com",
  "name": "Gopi"
}
```



8. Auth.js asks the adapter:

"Have I already seen this Google account?"

This is where the adapter becomes useful.

Conceptually Auth.js does something like:

```
</> TypeScript

adapter.getUserByAccount({
  provider: "google",
  providerAccountId: "987654321"
});
```



You generally don't write this call yourself.

Auth.js internally uses the adapter.

The Prisma adapter translates that into a database query.

Conceptually:

```
Auth.js
  ↓
```



```
Prisma Adapter
```

```
↓
```

```
Prisma
```

```
↓
```

```
PostgreSQL
```

9. Scenario A — user already exists

Suppose DB contains:

```
Account
```



```
-----  
userId          123  
provider        google  
providerAccountId 987654321
```

Adapter finds:

```
Google account
```



```
↓
```

```
userId = 123
```

Then Auth.js knows:

```
Google account
```



```
↓
```

```
Application user #123
```

10. Scenario B — first-ever login

Suppose this is Gopi's first login.

Database doesn't contain that Google account.

Auth.js can create the relevant records through the adapter.

Conceptually:

```
Auth.js
```



```
↓
```

```
adapter.createUser(...)
```

```
↓
```

```
Database
```

Creates:

User



```
-----  
id      123  
name    Gopi  
email   gopi@example.com
```

Then:

Auth.js



```
↓  
adapter.linkAccount(...)  
↓  
Database
```

Creates:

Account



```
-----  
userId      123  
provider    google  
providerAccountId 987654321
```

Now your application knows:

Google account #987654321



```
↓  
Application user #123
```

11. Now the SESSION

This is the part you were asking about earlier.

Auth.js now needs to establish:

"This browser is authenticated as user #123."

If you're using a **database session strategy**, Auth.js can create a session through the adapter.

Conceptually:

```
Auth.js
  ↓
adapter.createSession(...)
  ↓
Prisma Adapter
  ↓
PostgreSQL
```



Database:

```
Session
-----
sessionToken    userId    expires
XYZ_ABC_123     123      2026...
```



So now:

```
Session XYZ_ABC_123
      ↓
User 123
```



12. Auth.js gives the browser a cookie

Now Auth.js needs a way to identify this session on future requests.

It sends a cookie in the HTTP response.

Conceptually:

```
</> http

Set-Cookie: authjs.session-token=XYZ_ABC_123; HttpOnly; Secure; ...
```



The browser stores it.

So now:

```
Browser
-----
Cookie:
authjs.session-token
=
XYZ_ABC_123
```



Notice:

The browser doesn't need to know:

```
userId = 123
```



or:

```
password
```



It simply carries the session cookie.

13. Login is now complete 🎉

The whole first-login flow was:

```
Gopi
↓
Click Google
↓
Auth.js
↓
Google
↓
Google authenticates Gopi
↓
Callback
↓
Auth.js
↓
Adapter
↓
Create/find User
↓
Create Account
↓
Create Session
↓
Set Cookie
↓
Browser
```



14. Now Gopi visits /dashboard

This is where people often misunderstand authentication.

Gopi later requests:

```
</> http
```



```
GET /dashboard
```

```
Cookie: authjs.session-token=XYZ_ABC_123
```

The browser automatically sends the cookie.

You didn't manually attach it.

15. Server Component calls `auth()`

Your page:

```
</> TypeScript
```



```
// app/dashboard/page.tsx
```

```
import { auth } from "@auth";
```

```
import { redirect } from "next/navigation";
```

```
export default async function DashboardPage() {  
  const session = await auth();
```

```
  if (!session?.user) {  
    redirect("/login");  
  }
```

```
  return (  
    <h1>  
      Welcome {session.user.name}  
    </h1>  
  );  
}
```

Now:

```
</> TypeScript
```



```
await auth()
```

asks Auth.js:

```
"Who is authenticated for this request?"
```

16. What happens inside `auth()` ?

Conceptually:

```
Request
  ↓
Cookie
  ↓
Auth.js
  ↓
Read session token
  ↓
"XYZ_ABC_123"
  ↓
Adapter
  ↓
Database
  ↓
Find Session
  ↓
userId = 123
  ↓
Find User
  ↓
Gopi
  ↓
session
```



Auth.js then gives your Server Component something like:

```
</> TypeScript

{
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```



So your code can do:

```
</> TypeScript

session.user.name
```



17. Notice the difference between login and later requests

Login


```
Google
↓
Auth.js
↓
Create/find user
↓
Create session
↓
Set cookie
```



Later request

```
Cookie
↓
Auth.js
↓
Find session
↓
Find user
↓
Return session
```



You don't create a new session every time the user visits a page.

The existing session is used until it expires/is invalidated, with behavior depending on the configured session strategy.

18. What happens when the user logs out?

Your UI can use:

```
</> TypeScript

"use client";

import { signOut } from "next-auth/react";

export function LogoutButton() {
  return (
    <button onClick={() => signOut()}>
      Logout
    </button>
  );
}
```



Conceptually:

```
Browser
  ↓
signOut()
  ↓
Auth.js
  ↓
Invalidate/delete session
  ↓
Clear session cookie
```



If using database sessions, Auth.js can remove/invalidate the corresponding session through the adapter.

Then:

```
Browser
  ↓
GET /dashboard
  ↓
No valid session
  ↓
Not authenticated
  ↓
Redirect /login
```



19. Now let's answer your "WHERE IS THE CODE?" question precisely

You were asking:

"Where is the code that actually creates the session?"

Your code:

```
</> TypeScript

NextAuth({
  adapter: PrismaAdapter(prisma),
  providers: [
    Google(...)
  ]
});
```



Auth.js's internal code:

OAuth callback handling



Find/create user



Find/link account



Create session



Set session cookie



Read session



Expire/invalidate session



Adapter's implementation:

```
createUser()
getUser()
getUserByAccount()
linkAccount()
createSession()
getSessionAndUser()
deleteSession()
...
```

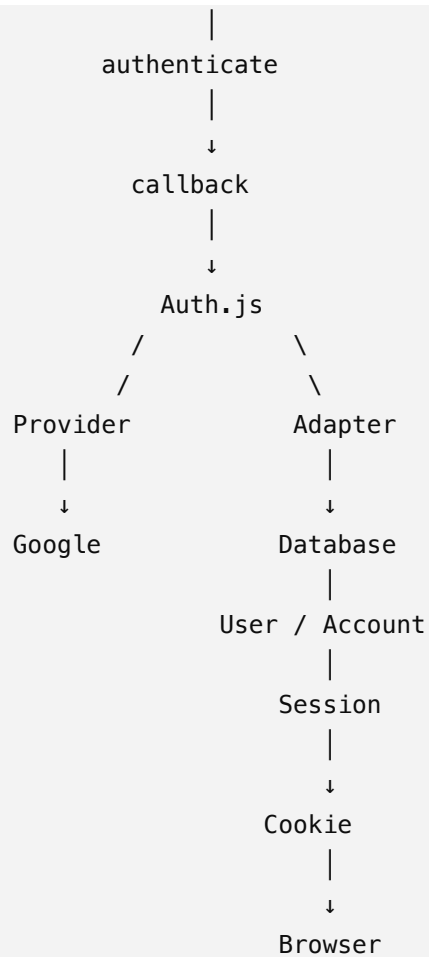


The exact internal implementation/API details can vary by Auth.js version and adapter, but **this is the architecture**.

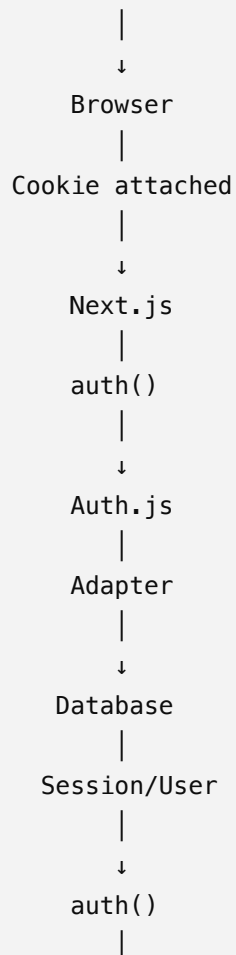
You are configuring the machinery rather than implementing the machinery yourself.

20. One final diagram — this is the one to memorize 🧠





EVERY LATER REQUEST



↓
Server Component

The four words to lock in:

Provider → *authenticates/identifies*

Auth.js → *orchestrates the auth flow*

Adapter → *connects Auth.js to persistence*

Database → *stores the persistent records*

And **session cookie** → *lets the browser prove/identify its existing authenticated session on subsequent requests.*

That's the whole machine. 🔥



Awesome 😄 Let's move to the **next important Auth.js concept**: how the session is actually used to protect pages and APIs.

Lesson 7.x — Protecting Routes with Auth.js

We've now understood:

Google



```
OAuth/OIDC
  ↓
Auth.js callback
  ↓
User/Account
  ↓
Session
  ↓
Cookie
```

Now the natural question is:

Once the user is logged in, how do I actually stop an unauthenticated user from accessing `/dashboard` or `/api/orders` ?

This is where `auth()` becomes really important.

1. Protecting a Server Component

Suppose we have:

```
app/
├─ login/
│   └─ page.tsx
└─ dashboard/
    └─ page.tsx
```

Our dashboard should only be accessible to logged-in users.

We can write:

```
<> TypeScript

// app/dashboard/page.tsx

import { auth } from "@auth";
import { redirect } from "next/navigation";

export default async function DashboardPage() {
  const session = await auth();

  if (!session) {
    redirect("/login");
  }

  return (
    <div>
```

```
    <h1>Welcome {session.user?.name}</h1>
    <p>{session.user?.email}</p>
  </div>
);
}
```

Now let's understand **exactly what happens**.

2. User requests `/dashboard`

Browser:

```
</> http
GET /dashboard
Cookie: authjs.session-token=ABC123
```

Next.js receives this request.

Because `dashboard/page.tsx` is a **Server Component**, it executes on the server.

Then:

```
</> TypeScript
const session = await auth();
```

runs.

3. What does `auth()` do?

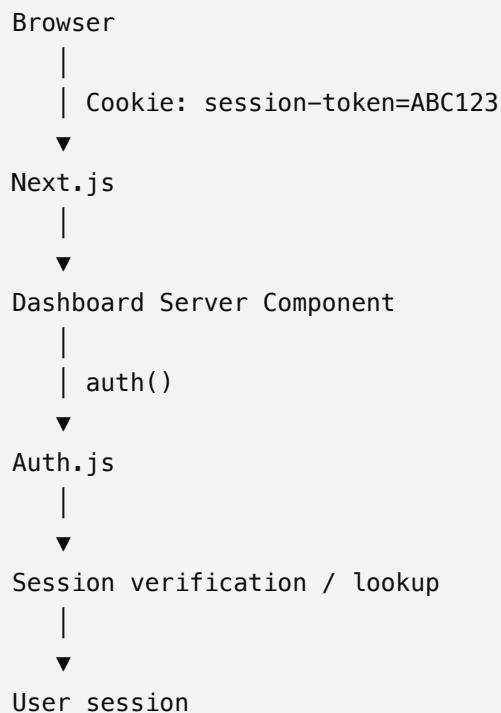
Remember where it came from:

```
</> TypeScript
// auth.ts

export const {
  handlers,
  auth,
  signIn,
  signOut
} = NextAuth({...});
```

So `auth()` is an Auth.js server-side function.

Conceptually:



If the session is valid:

`</> TypeScript`

```
session = {
  user: {
    name: "Gopi",
    email: "gopi@example.com"
  }
}
```

Then the page renders.

4. What if the user isn't logged in?

Suppose there is no valid session.

Then:

`</> TypeScript`

```
const session = await auth();
```

returns something equivalent to:

```
</> TypeScript
```



```
null
```

So:

```
</> TypeScript
```



```
if (!session) {  
  redirect("/login");  
}
```

executes.

Next.js sends a redirect response.

Conceptually:

```
</> http
```



```
307 Temporary Redirect  
Location: /login
```

The browser then requests:

```
</> http
```



```
GET /login
```

So:

```
User → /dashboard
```



```
  ↓  
auth()  
  ↓  
no session  
  ↓  
redirect("/login")  
  ↓  
/login
```

5. Why isn't this authorization?

This is an important distinction.

This code checks:

</> TypeScript



```
if (!session)
```

That is **authentication**.

We're asking:

"Is this person logged in?"

But suppose we have:

Admin Dashboard



and only admins can access it.

Then we need **authorization**.

For example:

</> TypeScript



```
const session = await auth();

if (!session) {
  redirect("/login");
}

if (session.user.role !== "ADMIN") {
  redirect("/forbidden");
}
```

Now we're asking two different questions:

Authentication:

"Who are you / are you logged in?"

Authorization:

"Are you allowed to do this?"



6. But there's an important security rule

Never rely only on hiding UI.

For example:

```
</> TypeScript

{session?.user.role === "ADMIN" && (
  <button>Delete User</button>
)}
```



This is useful for UX.

But it is **not security**.

A malicious user can manually send a request to your server.

For example:

```
</> http

POST /api/users/delete
```



Therefore your server-side action/API must check authorization too.

7. Protecting a Route Handler

Suppose we have:

```
app/api/orders/route.ts
```



We can do:

```
</> TypeScript

import { auth } from "@auth";
import { NextResponse } from "next/server";

export async function POST(request: Request) {
  const session = await auth();

  if (!session) {
    return NextResponse.json(
      { error: "Unauthorized" },
      { status: 401 }
    );
  }

  // User is authenticated
  // Continue processing order...

  return NextResponse.json({
```



```
    success: true,  
  });  
}
```

Now:

```
Browser  
  |  
  | POST /api/orders  
  | Cookie: session-token=ABC  
  ▼  
Next.js Route Handler  
  |  
  ▼  
auth()  
  |  
  ├── no session → 401  
  |  
  └── valid session → continue
```

8. This is why `auth()` is so useful

You can use the same server-side authentication context in multiple places:

```
Auth.js  
  |  
  auth()  
  |  
  ├──  
  ├──  
  └──  
Server Component  Route Handler  Server Action
```

For example:

Server Component

```
</> TypeScript  
  
const session = await auth();
```

Route Handler

```
</> TypeScript  
  
const session = await auth();
```

Server Action

</> TypeScript



```
const session = await auth();
```

The exact mechanics differ depending on the execution context, but the idea is:

Your server-side code asks Auth.js to resolve the current authenticated user/session.

9. Server Action example

Suppose:

</> TypeScript



```
"use server";

import { auth } from "@auth";

export async function deleteAccount() {
  const session = await auth();

  if (!session) {
    throw new Error("Unauthorized");
  }

  // Delete account...
}
```

Even if your UI hides the "Delete Account" button from logged-out users, the Server Action still checks authentication.

That's the security boundary.

10. Interview answer 🎯

If an interviewer asks:

"How would you protect a page in Next.js using Auth.js?"

A strong answer would be:

"For server-rendered pages, I can call Auth.js's server-side `auth()` function to resolve the current session. If there's no valid session, I redirect the user to the login page. For protected mutations or APIs, I perform the authentication and authorization checks again inside the Server Action or Route Handler rather than relying on client-side UI checks."

And if they ask:

"Is hiding an admin button authorization?"

Say:

"No. That's only a UI concern. Authorization must be enforced on the server at the resource or mutation boundary."

One thing I want you to notice

We now have **three different places** where Auth.js can participate:

1. LOGIN

```
signIn()  
  ↓  
OAuth  
  ↓  
callback  
  ↓  
session created
```



2. READING SESSION

```
auth()  
  ↓  
"Who is currently logged in?"
```

3. PROTECTING ACTIONS

```
auth()  
  ↓  
authenticate  
  ↓  
authorize  
  ↓  
perform operation
```

That distinction is **very interview-important**.

Next after this

The natural next piece is **Auth.js callbacks** — specifically:

`</> TypeScript`



```
callbacks: {
  async jwt() {},
  async session() {},
  async signIn() {},
  async redirect() {}
}
```

Because you've now seen `session` being created and consumed, the next question is:

"How do I customize what gets stored in the JWT/session — for example, how do I put a user's `role` or `id` into the session?"

That's where the `jwt()` and `session()` callbacks finally make sense. 🔥



how many more are there

Not too many 😊 We're actually pretty close to wrapping the **Authentication & Authorization** section.

Based on what we've covered so far, I'd structure the remaining part like this:

Remaining Auth.js / Auth & Authorization lessons

#	Topic	What you'll learn
7.6	Route Protection	<code>auth()</code> , protecting pages, APIs, Server Actions
7.7	Auth.js Callbacks	<code>signIn</code> , <code>jwt</code> , <code>session</code> , <code>redirect</code> callbacks
7.8	Session Customization	Put <code>user.id</code> , <code>role</code> , etc. into the session
7.9	Authorization / RBAC	Roles, permissions, admin/user access
7.10	Session Strategies	Database sessions vs JWT sessions

#	Topic	What you'll learn
7.11	 CSRF + SameSite	Why state-changing requests need protection
7.12	 Middleware / Proxy Auth	Protecting groups of routes before rendering
7.13	 Logout & Session Expiry	Sign-out, expiration, invalid/expired sessions
7.14	 Complete Auth Architecture	Put Browser → Next.js → Auth.js → OAuth → DB → Session together
7.15	 Auth Interview Questions	Real interview scenarios + architecture questions

So **about 10 lessons remain**, depending on how deeply we split things.

But some of these are short. The **big ones** are:

1. Callbacks
2. Session customization
3. Authorization/RBAC
4. JWT vs DB sessions
5. CSRF/SameSite
6. Middleware/Proxy

After that, I'd consider the Auth section **complete** and move you to the next major Next.js/interview area. 🚀

And I wouldn't rush through these—especially callbacks and authorization. Given the way you've been learning, I'd rather make sure you can explain the **actual request flow** than just memorize Auth.js APIs.



so after this authentication and authorization are done?

Yes. 👍 After we finish those remaining Auth.js topics, we can consider the **Authentication & Authorization section complete**.

More precisely, your **Next.js Auth & Authorization roadmap** will cover:

AUTHENTICATION



- ✓ Authentication vs Authorization
- ✓ Auth architecture
- ✓ Cookies
- ✓ Sessions
- ✓ JWT
- ✓ Password hashing
- ✓ Login flow
- ✓ Server Components + auth
- ✓ Client Components + auth
- ✓ Auth.js
- ✓ OAuth 2.0
- ✓ OpenID Connect
- ✓ Google authentication
- ✓ Authorization code + callback
- ✓ Client ID / Client Secret
- ✓ Token endpoint
- ✓ Auth.js adapters
- ✓ ORM + database persistence
- ✓ User / Account / Session
- ✓ Database sessions vs JWT sessions
- ✓ Route protection
- ✓ Auth.js callbacks
- ✓ Session customization

AUTHORIZATION

- ✓ Authentication vs authorization
- ✓ Protecting Server Components
- ✓ Protecting Route Handlers
- ✓ Protecting Server Actions
- ✓ Roles / RBAC
- ✓ Permissions
- ✓ Server-side authorization
- ✓ Client UI vs actual security boundary

SECURITY

- ✓ HttpOnly / Secure / SameSite
- ✓ CSRF
- ✓ Session expiry
- ✓ Logout
- ✓ Middleware / Proxy protection
- ✓ Common auth vulnerabilities

Then we'll do **one complete end-to-end architecture walkthrough**:



```
graph TD
    Browser --> Login
    Login --> Authjs[Auth.js]
    Authjs --> Google[Google OAuth/OIDC]
    Google --> Callback
    Callback --> Token[Google token exchange]
    Token --> User[User + Account]
    User --> Adapter
    Adapter --> Database
    Database --> Session
    Session --> Cookie
    Cookie --> Request[Protected request]
    Request --> Auth[auth()]
    Auth --> Authn[Authentication]
    Authn --> Authz[Authorization]
    Authz --> Resource[Protected resource]
```

That final walkthrough is important because you've been asking the right kind of **"but who actually calls what?"** questions. 😊 It will tie all the individual pieces together.

So yes: once we finish these, you can confidently mark *Next.js Authentication & Authorization* as DONE. 🗝️✅



ChatGPT can make mistakes. Check important info.